Automated Privacy Policy Compliance Verification Using Dynamic Network Traffic Analysis

Vimlesh Mourya
Department of Computer Science
SIES College of Arts, Science & Commerce (Autonomous)
Mumbai, India
Vimleshfmcs2526111@ascs.sies.edu.in

Maya Nair
Department of Computer Science
SIES College of Arts, Science & Commerce (Autonomous)
Mumbai, India
mayan@sies.edu.in

***Abstract*—**Privacy policies explain what information an application may collect, but they do not show what the application actually sends during use. This paper presents PrivacyComplianceVerifier, a framework that compares privacy-related evidence observed in HTTP Archive (HAR) traffic with explicit statements represented from an application's privacy policy. The framework preserves repeated headers, cookies, query parameters, request and response bodies, and supports common formats including JSON, multipart content, Base64, compressed text, raw Protocol Buffers, and gRPC frames. Detected values are redacted before an evidence inventory is generated. A conservative semantic layer then compares observed information types with reviewed policy evidence, while URL exposure and technical authentication artifacts remain separate risk signals. The system was evaluated on captures from 30 applications. It processed 34,170 network requests and identified 84,692 sensitive-data occurrences representing 5,354 unique artifacts. In the presentation-oriented result model, 13 applications had at least one comparable observed information type not represented in the reviewed policy evidence, 10 had all comparable observed types disclosed, and 7 could not be fully checked because policy evidence was incomplete. The results demonstrate that dynamic network evidence can make privacy-policy analysis more explainable, while also showing that a traffic capture is limited by the tested workflow, encryption, policy wording, and traffic ownership.

***Index Terms*—**privacy policy, dynamic analysis, network traffic, HAR, sensitive data, policy verification, Android applications.

I. INTRODUCTION

Mobile and web applications regularly process names, phone numbers, locations, device identifiers, account credentials, and other information. Users normally learn about this processing through a privacy policy. However, a policy is a written declaration, while the application's network traffic is observable runtime behaviour. The two sources can disagree because the policy is incomplete, the capture covers only a small workflow, or the implementation changes after the policy is written. Checking both sources manually is slow, especially when one capture contains thousands of requests.

Previous systems have compared mobile-app data flows with policy text. PoliCheck performs entity-sensitive consistency analysis between flows and privacy policies [1], while PurPliance focuses on the purposes for which apps use data [2]. OVRseen applies a similar idea to virtual-reality applications and highlights the difficulty of matching network evidence to natural-language policies [3]. More recent work uses program analysis, graph representations, and automated policy checking to find disclosure gaps [4]–[6]. These studies show that the main challenge is not only detecting data, but also keeping the meaning and source of each observation clear.

This project develops an explainable pipeline for college-level empirical analysis. It accepts HAR captures, extracts traffic without silently discarding duplicate fields, detects privacy-related artifacts in multiple body formats, classifies traffic ownership, and stores redacted evidence. It then compares only supported information types against reviewed policy evidence. The output avoids a legal claim of compliance. Instead, it reports whether an observed information type is disclosed, not disclosed in the available policy representation, or cannot be decided from the available evidence.

The main contributions of this work are:

- A duplicate-preserving HAR extraction pipeline for requests, responses, headers, cookies, query parameters, and bodies.
- A conservative detector that supports structured text, multipart content, encoded bodies, raw Protocol Buffers, and gRPC without exposing raw values in reports.
- An evidence-preserving inventory and semantic policy comparison that separates disclosure gaps from technical and URL-risk signals.
- An empirical study of 30 applications containing 34,170 requests and 84,692 detected sensitive occurrences.

II. RELATED WORK

A. Traffic and Policy Consistency

PoliCheck introduced an entity-sensitive method for checking whether observed data flows agree with privacy policies [1]. Its important idea is that the data type alone is not enough; the destination entity also affects the interpretation. PurPliance extended consistency analysis to the purposes described in policies [2]. OVRseen audited virtual-reality traffic and policies, demonstrating that nontraditional platforms also expose privacy evidence through network communication [3]. GNChecker later examined whether Android data transmissions were properly notified [4], and mopri combined app analysis and policy checks in a newer mobile setting [5].

These systems motivate the present framework, but this work uses a narrower and more reproducible rule: it does not infer a processing purpose from network traffic. The comparison is based on the observed information type, the available action and direction, and recipient information only when both sides support it. This prevents an apparently precise result from being based on unsupported assumptions.

B. Automated Privacy-Policy Analysis

Policy text is long and uses broad language. Amaral et al. studied automated completeness checking [7]. PoliGraph represents policy statements as knowledge graphs [6], while Rodríguez Torrado et al. explored large language models for analysis at scale [8]. A systematic mapping study by del Alamo et al. shows that automated policy analysis includes classification, information extraction, readability, and compliance tasks [9]. Other work has applied deep learning, rules, and machine learning to GDPR-related policy checks [10]–[12]. Viopolicy-Detector examines suspected GDPR violations on websites [23], while Farhadi et al. study automated application compliance checking in cloud and fog environments [24].

The present project does not treat automatic text extraction as complete ground truth. Policy statements are kept with provenance and a coverage state. When the policy representation is partial, absence of a practice is shown as an incomplete check instead of being forced into a mismatch. This design is important because a parser failure should not be presented as an application failure.

C. Network-Traffic Analysis

TraceDroid provides robust network analysis for privacy leakage in Android applications [13]. Li et al. use self-labelling and learning for privacy-traffic detection [14]. Surveys of encrypted traffic analysis describe both the value of network features and the challenge created by encryption [15], [16]. METAseen applies traffic and policy analysis to metaverse systems [17], and contextual-integrity work studies whether data flows are appropriate within a stated context [18]. Machine-learning classification [25], traffic-privacy optimization [26], policy-based protection [27], and edge data-stream controls [28] provide related approaches for protecting network information. These studies support dynamic observation, but they also warn that a capture is never a complete description of all app behaviour.

TABLE I. COMPARISON WITH SELECTED RELATED SYSTEMS

Approach	Dynamic traffic	Policy comparison	Recipient-aware	Evidence output
PoliCheck [1]	Yes	Yes	Yes	Flow/policy consistency
PurPliance [2]	App data use	Yes	Limited	Purpose consistency
OVRseen [3]	Yes	Yes	Yes	VR-labelled flows
GNChecker [4]	Yes	Yes	Notification focus	Undeclared transmission
This work	HAR requests + responses	Yes	Conservative	Redacted occurrence evidence

III. PROPOSED FRAMEWORK

PrivacyComplianceVerifier is organized as six stages, as shown in Fig. 1. HTTP Toolkit is used to capture decrypted application traffic and export it as HAR. The extractor converts HAR entries into an internal request model. A traffic classifier separates application-owned, third-party, Android-system, and unknown destinations. The sensitive-data detector scans supported request and response locations. Findings are normalized and stored in a privacy inventory. Finally, the comparator checks observed information types against policy evidence and reports disclosure results and independent risk signals.

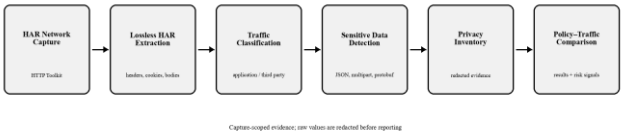


Fig. 1. PrivacyComplianceVerifier processing pipeline.

A. Lossless HAR Extraction

A normal dictionary can silently overwrite duplicate headers, cookies, or query keys. The extractor therefore keeps ordered pairs in addition to backward-compatible dictionary fields. It reads request and response headers, cookie arrays and raw cookie headers, URL query parameters, request post data, HTTP Toolkit request-content fields, response content, MIME types, declared charsets, and HAR encoding metadata. Parameters represented in both text and HAR arrays are reconciled so that the same underlying occurrence is not reported twice.

B. Sensitive-Data Detection

The detector searches known keys and validated value patterns for information such as email, phone, name, address, coordinates, device identifier, IP address, account identifier, authentication token, and session cookie. Nested JSON is traversed with depth and pair limits. Stringified JSON, structured query values, and structured cookie values are parsed only when they begin with a JSON object or array marker and pass bounded parsing. Arrays use the nearest nonnumeric parent segment as an additional exact key candidate.

Bodies may be plain text, Base64 or Base64url, compressed with supported encodings, multipart, raw protobuf, or gRPC-framed protobuf. Multipart parsing uses valid line-anchored boundaries and does not scan arbitrary binary file parts. Raw protobuf parsing extracts only length-delimited strings; it does not guess the meaning of numeric fields. Large free-text bodies are scanned in bounded overlapping chunks. These controls reduce crashes, excessive work, and false positives on opaque data.

C. Evidence and Privacy Inventory

Each normalized finding keeps its artifact type, privacy category, source, direction, key, domain, and traffic type. The raw detected value is not written to the privacy inventory. Repeated occurrences remain separate so frequency can be measured, while fingerprints prevent the same value reached through duplicate parser paths from being counted twice. The inventory also keeps request counts, domains, and categories for compatibility with the original project output.

D. Policy–Traffic Comparison

Observed data is summarized as a practice containing a privacy category, artifact type, action or direction, traffic recipient scope, destination domain, source, and risk flags. Policy statements are normalized separately and retain their source and review metadata. The system does not directly equate application-owned traffic with a legal first-party recipient. Recipient scope is compared only when the mapping is explicitly supported.

The presentation model uses three direct outcomes. An observed comparable information type is Disclosed when a reviewed policy statement explicitly covers it. It is Not Disclosed when the reviewed policy representation is complete for that comparison scope and no matching type exists. Otherwise, the result is Policy Check Incomplete. Authorization tokens and session cookies remain visible as technical evidence but do not create dashboard policy gaps. Security artifacts, API credentials, and unknown categories are also kept outside automatic mismatch logic. Separately, a personal value or credential in a URL is reported as a risk signal even when its type is disclosed.

IV. RESEARCH METHODOLOGY

A. Application Selection and Capture

The empirical dataset contains 30 widely used mobile and web applications, including communication, social, shopping, food-delivery, entertainment, productivity, navigation, employment, and open-source applications. Traffic was captured using HTTP Toolkit while the researcher performed common available functions such as login, search, profile access, browsing, messaging, media access, and location-related actions. The exact workflow varied because applications provide different features and some traffic could not be decrypted.

This is an activity-based sample, not an exhaustive test. A category that was not observed may still be processed during an untested action, in background execution, on another device, or inside traffic not visible to the capture tool. Therefore, 'not observed' means only that the evidence was absent from this capture.

B. Policy Evidence Preparation

The official privacy-policy page for each application was reviewed and represented in JSON. Legacy categories were preserved, and explicit statements were converted into canonical information-type practices with provenance. The policy publication date and the manual review date were kept separately. No recipient domain or action was added unless the policy text supported it. Policies with incomplete extraction were marked partial rather than complete. This avoids creating a non-disclosure only because a policy file lacks detail.

C. Measurements

The evaluation records total requests, traffic ownership, sensitive occurrences, unique artifacts, comparable outbound findings, attributable inbound findings, category counts, disclosure decisions, and URL-risk signals. Occurrence counts measure repeated network evidence. Unique-artifact counts collapse repeated representations of the same redacted value. Disclosure counts are based on distinct comparable information types rather than every network occurrence. This prevents a frequently repeated token or identifier from dominating the policy result.

V. IMPLEMENTATION

The framework is implemented in Python. Request extraction, traffic classification, detection, normalization, inventory generation, policy parsing, practice matching, risk assessment, reporting, and dashboard aggregation are separate modules. This separation makes it possible to test the detector without changing the comparator and to change the presentation without losing raw evidence.

The detector uses bounded parsing throughout. JSON recursion, stringified-JSON recursion, decoded size, multipart boundary length, decompressed output, protobuf parsing, and free-text scanning all have limits. Email matching uses candidate-bounded checks instead of an unbounded expression over an arbitrary body. Charset decoding is applied to bytes only; already decoded Python strings are not encoded and decoded again. These choices were added after regression tests exposed realistic failure cases.

The automated test suite contains 244 passing tests. Focused regression tests cover duplicate headers and cookies, query parameters, post-data merging, Base64 and Base64url, UTF-16 bodies, compressed content, large-text chunk boundaries, malformed multipart input, binary form parts, raw protobuf, multi-frame gRPC, repeated occurrences, inventory evidence, semantic mapping, and policy comparison. The project also provides JSON reports, aggregate CSV/JSON results, a command-line summary, and a local presentation dashboard.

VI. RESULTS AND DISCUSSION

A. Overall Dataset

Across 30 applications, the framework processed 34,170 HAR requests. It detected 84,692 sensitive occurrences and 5,354 unique sensitive artifacts. Of the findings, 44,490 were comparable outbound occurrences and 34,178 were attributable inbound occurrences. The large difference between occurrences and unique artifacts shows that the same identifiers and account information are often transmitted repeatedly.

TABLE II. EMPIRICAL DATASET SUMMARY

Measure	Value
Applications	30
HAR network requests	34,170
Sensitive occurrences	84,692
Unique sensitive artifacts	5,354
Comparable outbound occurrences	44,490
Attributable inbound occurrences	34,178
URL-risk signal records	35
Automated tests	244 passing

B. Observed Privacy Categories

Personal Information was the largest observed category with 27,073 occurrences. Authentication followed with 22,717 occurrences, while Location and Device Identifier produced 17,306 and 14,138 occurrences respectively. Security, API Credentials, and Network Information were less frequent. These counts describe detected traffic artifacts; they do not by themselves show whether an application's policy is adequate.

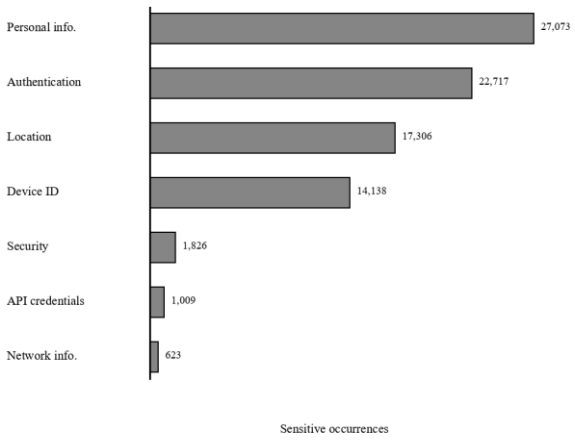


Fig. 2. Sensitive occurrences by privacy category.

C. Policy Comparison Results

The reviewed presentation result contained 70 disclosed information types and 24 observed comparable types that were not represented in completed policy evidence. Ten additional comparable type records could not be decided because the policy check was incomplete. At application level, 13 applications had at least one potential non-disclosure, 10 had all comparable observed types disclosed, and 7 had an incomplete policy check.

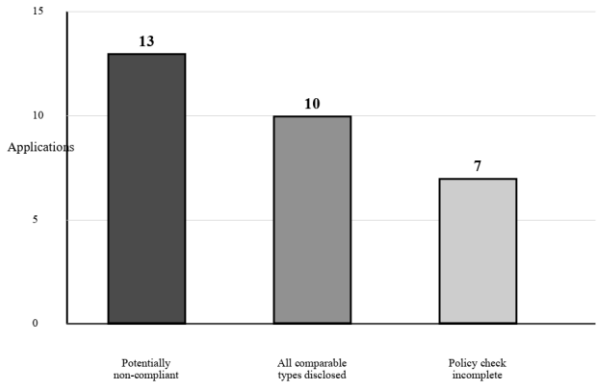


Fig. 3. Application-level policy–traffic result distribution.

The phrase 'potentially non-compliant' is used as a project result label, not as a legal judgment. It means that a comparable information type was observed in application-owned outbound traffic and was absent from a policy representation reviewed as complete for that scope. A disclosed result also does not prove complete compliance; it only indicates that the observed type was represented in the available policy evidence.

D. URL Risk Signals

Nineteen applications produced personal-data-in-URL signals and sixteen produced credential-in-URL signals, for 35 signal records in total. URL evidence is shown separately because a disclosed data type can still be exposed in a risky request location. URLs may be stored in browser history, access logs, proxies, analytics systems, and screenshots. The signal therefore asks for technical review; it is not automatically converted into a policy mismatch. Network-traffic capture and storage can itself create privacy risks, so reports contain redacted values rather than the original personal data [19].

E. Truecaller Case Study

Truecaller provides a useful example of why disclosure and transport risk must be separated. The capture contained 184 requests, 216 sensitive occurrences, and eight unique artifacts. Outbound application traffic included phone numbers, names, email addresses, and authorization tokens. The reviewed policy evidence covered Phone, Name, and Email, so these comparable types were disclosed. Authorization Token remained visible technical evidence but was not treated as a missing privacy-policy item. At the same time, two personal-data-in-URL evidence items produced a medium risk signal. Thus the policy can mention the information while the transport location still deserves review.

TABLE III. TRUECALLER OUTBOUND OBSERVATIONS

Artifact	Occurrences	Source	Dashboard treatment
Authorization Token	107	Header	Technical evidence
Phone	67	Query / gRPC	Disclosed
Name	8	Request body	Disclosed
Email	4	Request body	Disclosed
Personal data in URL	2 signals	Query/URL	Risk review

F. External Dataset Experiment

Small experiments were also performed with traffic from AntShield, PARROT 2025, and OVRseen. These experiments were used to check format portability, not to claim benchmark accuracy. For example, one converted OVRseen RecRoom sample produced seven requests, ten sensitive occurrences, and three unique artifacts. An AntShield Facebook sample produced 99 requests, 234 occurrences, and nine unique artifacts. The exercise showed that the detector can consume externally sourced HTTP evidence when it can be represented faithfully as HAR. Labels from external datasets remain dataset-provided evidence rather than absolute ground truth.

VII. LIMITATIONS AND THREATS TO VALIDITY

A. Capture Coverage

The strongest limitation is workflow coverage. A manual session cannot activate every feature, background job, account state, error path, regional option, or advertisement. A 'not observed' result is therefore not evidence that the application never processes that information. Repeating the capture with scripted workflows and several accounts would improve reproducibility.

B. Encryption and Representation

Some applications use certificate pinning, custom encryption, QUIC, or binary formats for which only part of the meaning is visible. Raw protobuf support extracts strings without a schema and intentionally does not infer numeric fields. Brotli, Zstandard, bodies beyond strict safety limits, images, PDFs, and video are not semantically inspected. These restrictions can cause false negatives, but removing them without controls could create false positives and unsafe resource use. Encrypted-traffic research shows that useful classification is possible, but interpreting content without decryption remains difficult [15], [16].

C. Policy Interpretation

Policies use broad legal terms, while traffic contains engineering terms such as field names and headers. Mapping these vocabularies requires judgment. The framework does not map Device Identifier to Device Information or Authentication to Passwords unless the policy evidence supports the exact type. Conversely, a policy may cover a type with wording that was not represented in the project JSON. Partial coverage is therefore reported as an incomplete check. Studies of policy automation report the same difficulty in converting natural-language statements into complete structured representations [7]–[9]. Privacy-aware cloud auditing [29] and mobile privacy-risk assessment [30] also show that evidence quality and context affect the result.

D. Traffic Ownership and Generalisation

Domains were classified using app configuration and known ownership rules. Content-delivery networks, shared cloud infrastructure, redirects, and embedded services can still be ambiguous. An application-owned domain is not automatically the same as a policy-defined first party. The selected 30 applications are useful case studies but do not represent all Indian or global applications. Results also depend on the app versions, dates, accounts, and network conditions used during collection.

E. Ethics and Data Handling

HAR files can contain personal values and credentials. The raw dataset is therefore kept outside the public Git repository. Reports use redacted values, and the paper presents only aggregate counts and non-sensitive examples. Future work should add stronger automated secret scanning and a documented retention procedure for raw captures.

This study was carried out only for academic research. The results describe technical observations from specific app versions tested in a controlled environment. They show the data-handling behaviour seen during those captures and are not a legal judgment or final proof of non-compliance. Different app versions, user actions, devices, or network conditions may produce different results.

VIII. CONCLUSION AND FUTURE WORK

This paper presented PrivacyComplianceVerifier, a dynamic framework for comparing privacy-related network evidence with application privacy policies. The framework improves HAR extraction, supports several structured and binary formats, preserves evidence provenance, and separates policy results from transport-risk signals. Evaluation on 30 applications showed that large amounts of personal, authentication, location, and device information can be made visible and summarized without publishing raw values. The result model found disclosed types, potential disclosure gaps, incomplete policy checks, and risky URL placements.

The framework does not prove legal compliance and cannot observe behaviour outside the captured session. Future work should automate repeatable interaction workflows, improve policy extraction with human verification, add schema-aware protobuf analysis where schemas are available, measure precision and recall on a suitable labelled dataset, and study traffic ownership in shared infrastructure. These improvements would make the evidence stronger without turning uncertain observations into artificial violations.

REFERENCES

[1] B. Andow, S. Y. Mahmud, J. Whitaker, W. Enck, B. Reaves, K. Singh, and S. Egelman, “Actions Speak Louder than Words: Entity-Sensitive Privacy Policy and Data Flow Analysis with PoliCheck,” 2020.

[2] D. Bui, Y. Yao, K. Shin, J.-M. Choi, and J. Shin, “Consistency Analysis of Data-Usage Purposes in Mobile Apps,” in Proc. ACM SIGSAC Conf. Computer and Communications Security, 2021, doi: 10.1145/3460120.3484536.

[3] R. Trimananda, H. Le, H. Cui, J. Ho, A. Shuba, and A. Markopoulou, “Auditing Network Traffic and Privacy Policies in Oculus VR,” 2021.

[4] M. Fan, J. Shi, Y. Wang, L. Yu, X. Zhang, H. Wang, W. Jin, and T. Liu, “Giving without Notifying: Assessing Compliance of Data Transmission in Android Apps,” in Proc. IEEE/ACM Int. Conf. Automated Software Engineering, 2024, doi: 10.1145/3691620.3695528.

[5] C. Ziepel, S. Escher, S. Rehms, and S. Köpsell, “mopri—An Analysis Framework for Unveiling Privacy Violations in Mobile Apps,” arXiv, 2026, doi: 10.48550/arxiv.2602.03671.

[6] H. Cui, R. Trimananda, and A. Markopoulou, “PoliGraph: Automated Privacy Policy Analysis using Knowledge Graphs,” arXiv, 2022, doi: 10.48550/arxiv.2210.06746.

[7] O. Amaral, S. Abualhaija, D. Torre, M. Sabetzadeh, and L. Briand, “AI-Enabled Automation for Completeness Checking of Privacy Policies,” IEEE Trans. Software Engineering, 2021, doi: 10.1109/TSE.2021.3124332.

[8] D. Rodríguez Torrado, I. Yang, J. M. del Álamo, and N. Sadeh, “Large language models: a new approach for privacy policy analysis at scale,” Computing, 2024, doi: 10.1007/s00607-024-01331-9.

[9] J. M. del Alamo, D. S. Guamán, B. García, and A. Diez, “A systematic mapping study on automated analysis of privacy policies,” Computing, 2022, doi: 10.1007/s00607-022-01076-3.

[10] T. A. Rahat, M. Long, and Y. Tian, “Is Your Policy Compliant? A Deep Learning-based Empirical Study of Privacy Policies’ Compliance with GDPR,” in Proc. Workshop on Privacy in the Electronic Society, 2022, doi: 10.1145/3559613.3563195.

[11] S. Liu, B. Zhao, R. Guo, G. Meng, F. Zhang, and M. Zhang, “Have You been Properly Notified? Automatic Compliance Analysis of Privacy Policy Text with GDPR Article 13,” in Proc. Web Conf., 2021, doi: 10.1145/3442381.3450022.

[12] R. El Hamdani, M. Mustapha, D. Restrepo Amariles, A. Troussel, S. Meeûs, and K. Krasnashchok, “A combined rule-based and machine learning approach for automated GDPR compliance checking,” in Proc. Int. Conf. Artificial Intelligence and Law, 2021, doi: 10.1145/3462757.3466081.

[13] H. Cui, G. Meng, Y. Zhang, W. Wang, D. Zhu, T. Su, X. Zhang, and Y. Li, “TraceDroid: A Robust Network Traffic Analysis Framework for Privacy Leakage in Android Apps,” 2022, doi: 10.1007/978-3-031-17551-0_35.

[14] Y. Li, H. Cui, J. Sun, Y. Zhang, Y. Li, G. Meng, and W. Wang, “Automated Privacy Network Traffic Detection via Self-labeling and Learning,” in Proc. IEEE Symp. Computers and Communications, 2022, doi: 10.1109/ISCC55528.2022.9912471.

[15] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, “Machine Learning-Powered Encrypted Network Traffic Analysis: A Comprehensive Survey,” IEEE Commun. Surveys & Tutorials, 2023, doi: 10.1109/COMST.2022.3208196.

[16] E. Papadogiannaki and S. Ioannidis, “A Survey on Encrypted Network Traffic Analysis Applications, Techniques, and Countermeasures,” ACM Computing Surveys, 2021, doi: 10.1145/3457904.

[17] B. Yu, Y. Liu, S. Ren, Z. Zhou, and J. Liu, “METAseen: analyzing network traffic and privacy policies in Web 3.0 based Metaverse,” Digital Communications and Networks, 2023, doi: 10.1016/j.dcan.2023.11.006.

[18] A. Markopoulou, R. Trimananda, and H. Cui, “A CI-based Auditing Framework for Data Collection Practices,” arXiv, 2023, doi: 10.48550/arxiv.2303.17740.

[19] M. Mastroianni, F. Palmieri, M. Ficco, R. Kozik, and M. Choraś, “Privacy risk analysis and metrics in capturing and storing network traffic,” in Proc. Int. Conf. Control Systems and Computer Science, 2023, doi: 10.1109/CSCS59211.2023.00097.

[20] T. Heino, R. Carlsson, S. Rauti, and V. Leppänen, “Assessing discrepancies between network traffic and privacy policies of public sector web services,” in Proc. Int. Conf. Availability, Reliability and Security, 2022, doi: 10.1145/3538969.3539003.

[21] H. Atapattu, W. Fernando, J. P. A. K. Somasiri, P. M. K. Lokuge, A. Senarathne, and M. Tissera, “A Sensitive Data Leakage Detection and Privacy Policy Analyzing Application for Android Systems (PriVot),” in Proc. Int. Conf. Advancements in Computing, 2021, doi: 10.1109/ICAC54203.2021.9671075.

[22] D. Sánchez, A. Viejo, and M. Batet, “Automatic Assessment of Privacy Policies under the GDPR,” Applied Sciences, vol. 11, 2021, doi: 10.3390/app11041762.

[23] H. Ou, Y. Fang, Y. Guo, W. Guo, and C. Huang, “Viopolicy-Detector: An Automated Approach to Detecting GDPR Suspected Compliance Violations in Websites,” in Proc. Int. Symp. Research in Attacks, Intrusions and Defenses, 2022, doi: 10.1145/3545948.3545952.

[24] M. Farhadi, G. Pierre, and D. Miorandi, “Towards automated privacy compliance checking of applications in Cloud and Fog environments,” in Proc. Int. Conf. Future Internet of Things and Cloud, 2021, doi: 10.1109/FiCloud49777.2021.00010.

[25] I. A. Alharbi, C. Zou, and R. N. Alharbi, “Encrypted Network Traffic Analysis and Classification Utilizing Machine Learning,” Sensors, vol. 24, 2024, doi: 10.3390/s24113509.

[26] L. Chaddad, A. Chehab, and A. Kayssi, “OPriv: Optimizing Privacy Protection for Network Traffic,” Journal of Sensor and Actuator Networks, vol. 10, 2021, doi: 10.3390/jsan10030038.

[27] G. Baldini, J. L. Hernández-Ramos, S. Nowak, R. Neisse, and M. Nowak, “Mitigation of Privacy Threats due to Encrypted Traffic Analysis through a Policy-Based Framework and MUD Profiles,” Symmetry, vol. 12, 2020, doi: 10.3390/sym12091576.

[28] B. Sedlak, I. Murturi, P. K. Donta, and S. Dustdar, “A Privacy Enforcing Framework for Data Streams on the Edge,” IEEE Transactions on Emerging Topics in Computing, 2024, doi: 10.1109/TETC.2023.3315131.

[29] M. Barati, G. Aujla, J. T. Llanos, K. A. Duodu, O. Rana, M. Carr, and R. Ranjan, “Privacy-Aware Cloud Auditing for GDPR Compliance Verification in Online Healthcare,” IEEE Transactions on Industrial Informatics, 2021, doi: 10.1109/TII.2021.3100152.

[30] D. Blanco-Aza, A. Robles-Gómez, R. Pastor Vargas, L. Tobarra, P. Vidal-Balboa, and M. Méndez-Suárez, “Comprehensive AI-driven privacy risk assessment in mobile apps and social networks,” Cluster Computing, 2025, doi: 10.1007/s10586-025-05624-2.